

Advanced Mobile Security • DAY 5

How Mobile Apps Get Broken & How They're Defended

Session 1: eight ideas, in plain language, each backed by real, documented incidents

Before We Start: One Big Idea

Everything today comes back to a single sentence: the phone in someone's hand is not a trusted place.



Today's 8 Ideas

Session 1 · 09:30–11:00 · 90 minutes

1

Why hacks are rarely "one trick"

attack chains

2

How an app can be secretly edited

tampering & repackaging

3

Changing an app while it runs

runtime manipulation

4

Why some code is hard to read

reverse engineering

5

Breaking the phone's "safe boxes"

root & jailbreak

6

Asking the chip, not the software

hardware attestation

7

When a "return link" gets hijacked

deep links

8

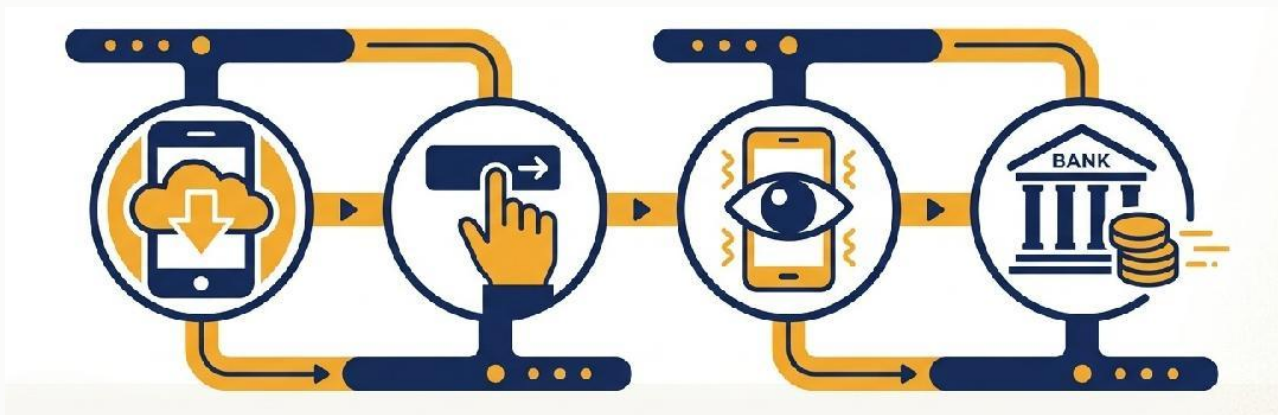
Putting it all on one checklist

threat modelling

Each idea gets: a picture, a plain-language explanation, and one or two real cases.

Why Hacks Are Rarely "One Trick"

Real break-ins chain a few small, boring tricks together - not one clever bug.





1 / 8

Why Hacks Are Rarely "One Trick"

Think of a burglar who finds an unlocked window, then a spare key inside, then the safe combination taped to a drawer. Mobile attackers work the same way - a few small steps, each one boring by itself.



- Step 1: a harmless-looking app gets installed - nothing malicious inside it yet.
- Step 2: it asks for "Accessibility" access - a real feature meant to help users with disabilities.
- Step 3: once granted, it can read your screen and tap buttons for you, unnoticed - even inside your banking app.



CASE 1

SharkBot banking trojan

First identified by security firm Cleafy in 2021. In 2022, researchers at NCC Group found a version hiding inside a fake "antivirus" app on the Google Play Store. Once installed, it used the Accessibility trick to wait for the victim to log into their real banking app, then quietly initiated its own money transfers - the victim's own genuine, logged-in session was used against them.

[NCC Group: SharkBot dropper in the Google Play Store](#)





CASE 2

Not a one-off: Xenomorph & Godfather

This isn't a single incident - it's a pattern. ThreatFabric documented Xenomorph spreading via a "Fast Cleaner" app on Google Play in 2022, targeting 56 European banks. Around the same time, Group-IB and ThreatFabric tracked Godfather, which used the same Accessibility trick against more than 400 banking and crypto apps worldwide.

[ThreatFabric: Xenomorph banking trojan report](#)



How an App Can Be Secretly Edited

Apps can be opened up, edited, and resealed - like a package that's been secretly tampered with.



SEALED BOX



OPEN & EDITING



**RESEALED
WITH STAMP**



2 / 8

How an App Can Be Secretly Edited

Imagine buying a sealed package, secretly opening it, swapping one part, and resealing it so well you'd never know it was touched. Attackers can do this to apps.



- The app is taken apart into a readable, editable form.
- One line - often a safety check - is changed or deleted.
- It's rebuilt, signed with the attacker's own certificate, and shared outside the official app store.



CASE

A modified WhatsApp, and the trojan hiding inside it

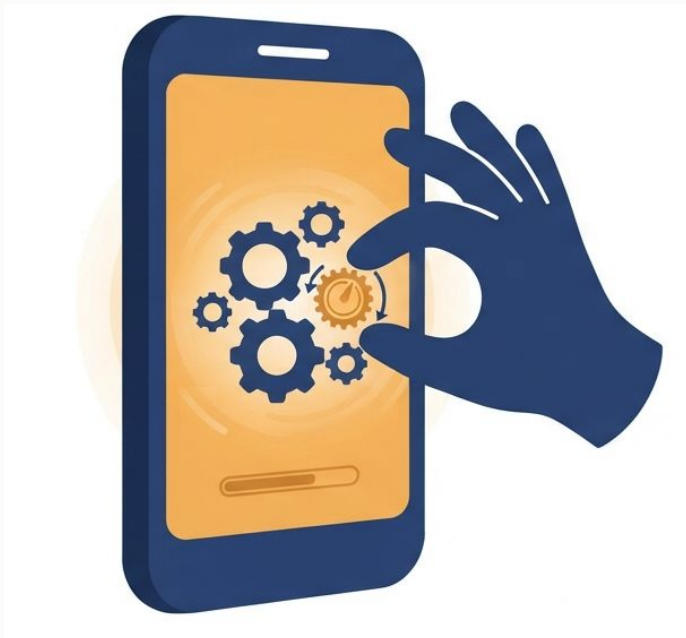
In 2021, Kaspersky found the Triada trojan bundled into a popular modified version of WhatsApp called FMWhatsApp - available only outside the official app stores. Because the mod genuinely delivered the extra features it promised, users had no reason to suspect anything. The bundled trojan could read SMS messages, sign users up for paid subscriptions, and hijack the WhatsApp account itself.

[Kaspersky Securelist: Triada trojan in a WhatsApp mod](#)



Changing an App While It Runs

Some tools change what an app does live, in memory - without ever touching the file on disk.

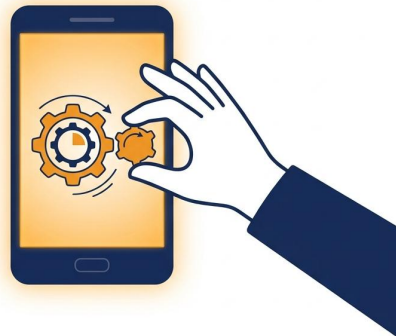




3 / 8

Changing an App While It Runs

Editing the file (idea 2) isn't the only option. Some tools change what an app does live, while it's running - like a puppeteer grabbing the strings mid-show, without ever touching the puppet.



- A tool called Frida can attach to an app that's already open on the phone.
- It can force one specific check to always say "yes" - e.g. a fake fingerprint approval.
- Nothing on disk changes, so simple file checks never notice anything happened.

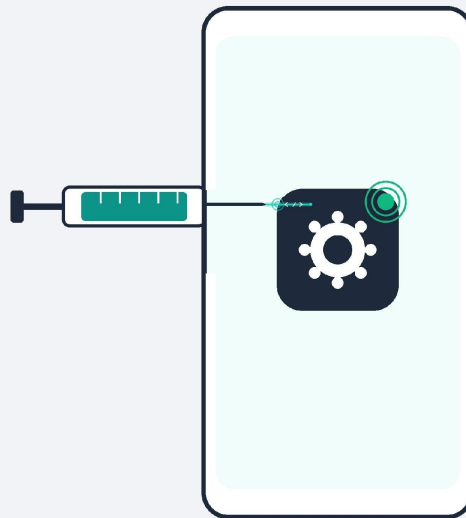


REFERENCE

Why OWASP treats this as its own defense category

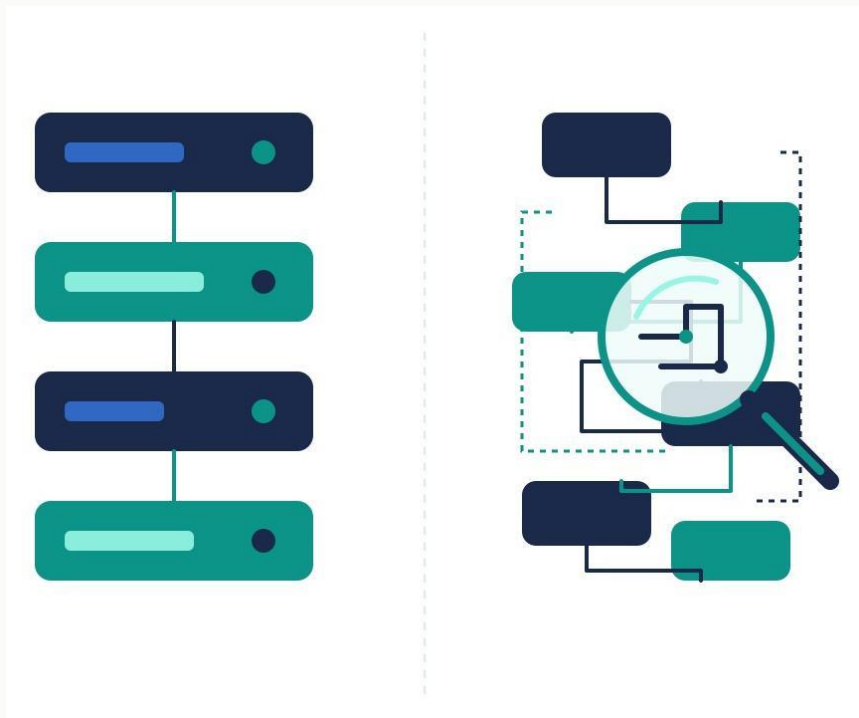
This exact technique is common enough that OWASP's official mobile security standard has a dedicated section just for resisting it - called "Resilience." It covers exactly the scenario we just described: an app whose logic can be silently overridden while it's running, and what developers can do to make that harder.

[OWASP: MASVS-RESILIENCE - resisting reverse engineering and tampering](#)



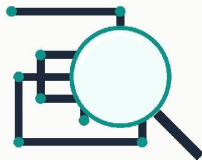
Why Some Code Is Hard to Read

An app is just instructions - if they're easy to read, an attacker can find secrets and weak spots too.





4 / 8



Why Some Code Is Hard to Read

An app is really just a set of instructions. If they're easy to read, anyone - including an attacker - can find secret keys or weak spots inside. So developers deliberately make code harder to follow.

- Easy step: rename everything to meaningless letters (a, b, c).
- Harder step: scramble the order of decisions so the flow looks tangled.
- Hardest step: move the most sensitive logic into a completely different, much harder-to-read format.



REFERENCE

A speed bump, not a wall

OWASP's official testing guide is explicit that this kind of protection is measured in "how much longer did it take an analyst," not "did it stop the analyst." It's the same logic as a home security system: it won't stop a determined attacker forever, but it makes your house a less attractive target than the house next door.

[OWASP MASTG: the mobile security testing guide](#)





Quick Breather - Halfway Through

So far: apps get broken by chaining small tricks (1), editing the binary (2), poking it while it runs (3), and reading code that wasn't meant to be read (4). Four more to go.

Breaking the Phone's "Safe Boxes"

Every app normally lives in its own locked box. "Rooting" breaks every box open at once.





5 / 8

Breaking the Phone's "Safe Boxes"



Your phone keeps every app in its own locked box, so apps can't peek at each other. "Rooting" (Android) or "jailbreaking" (iPhone) breaks every box open at once - usually done on purpose by the owner, but it removes protection for every app.

- Sensitive apps try to detect this by looking for a tell-tale file the rooting tool leaves behind.

- Newer tools (Magisk, KernelSU) hide that file so well the app can't find it at all.

- This turns "check for root" into a game the checker keeps losing.



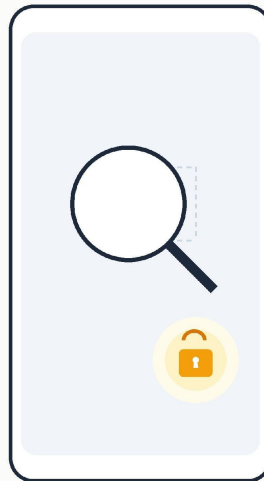
CASE

The ongoing hide-and-seek: Magisk and KernelSU

Magisk's own documentation explains how it leave the physical system partition untouched, so a simple file existence check finds nothing.

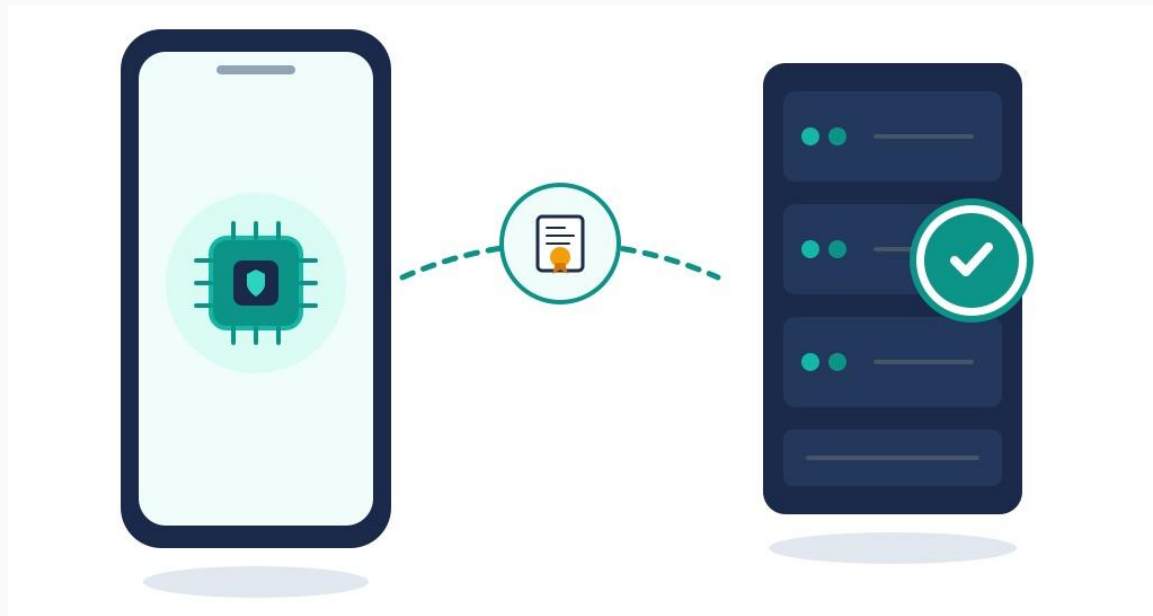
KernelSU goes a step further and runs inside the phone's kernel itself - the deepest layer of the operating system where a normal app has no way to look at all.

[Magisk documentation](#) [KernelSU documentation](#)



Asking the Chip, Not the Software

If software can lie about whether a phone is safe, ask a separate piece of hardware instead.

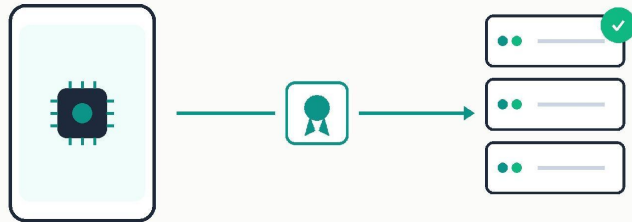




6 / 8

Asking the Chip, Not the Software

If an app's own software can be tricked - as we just saw - asking it "are you safe?" is pointless, because a compromised phone can simply lie. The fix: ask a separate, tamper-resistant chip instead.



- Android's Play Integrity API and Apple's App Attest ask a dedicated security chip, not the operating system.
- That chip produces proof that software alone can't forge.
- Only your own server should trust that proof - never the app itself (remember idea 3).

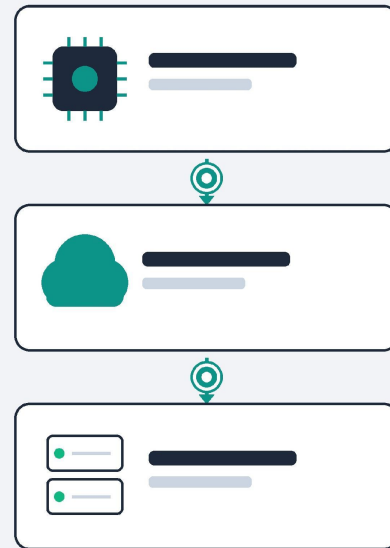


REFERENCE

Why the server has to be the one checking

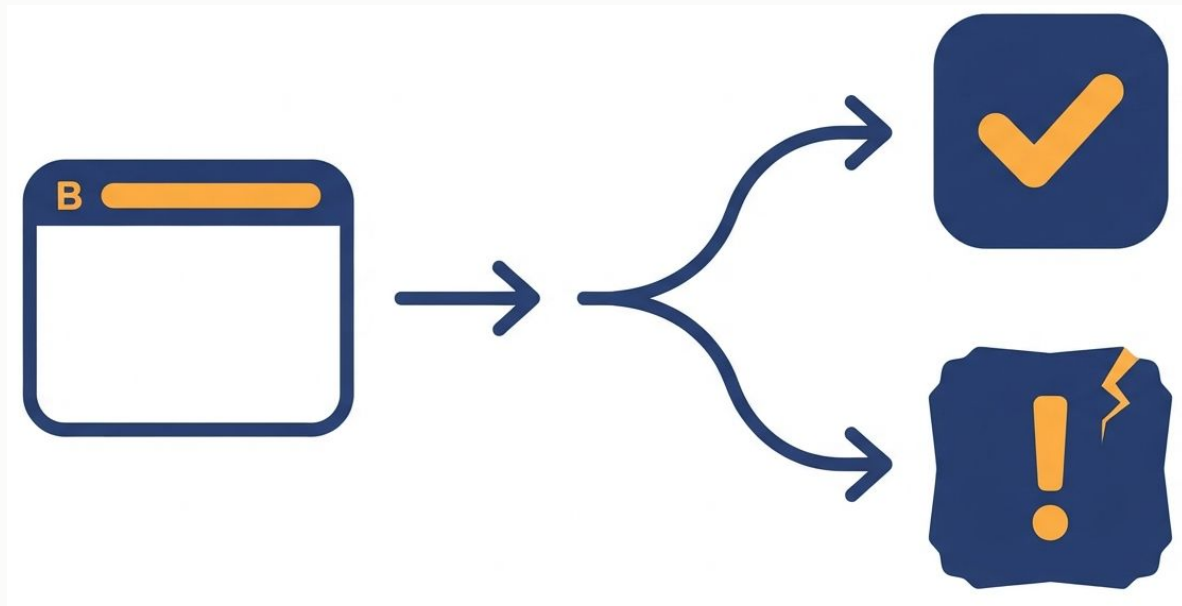
Google and Apple both publish this exact pattern as the intended design: the phone proves itself to its own chip, the chip proves it to Google or Apple's servers, and the result is handed to your backend as a signed token - which only your server verifies. The app on the phone never gets to make the final decision.

[Google Play Integrity API](#) [Apple App Attest](#)



When a "Return Link" Gets Hijacked

Apps use special "return address" links to bring you back after logging in - and those can be hijacked.





7 / 8

When a "Return Link" Gets Hijacked

When an app sends you to a login page and expects you to come back afterward, it uses a special "return address" link - but the phone doesn't always check who's really allowed to claim it.

- An unchecked link can be forced to open inside a powerful, privileged in-app browser instead of a normal one.
- If that in-app browser is connected to sensitive account features, a crafted link can hijack the account in one tap.
- The fix: links tied to a real, verified website domain - not a made-up address.

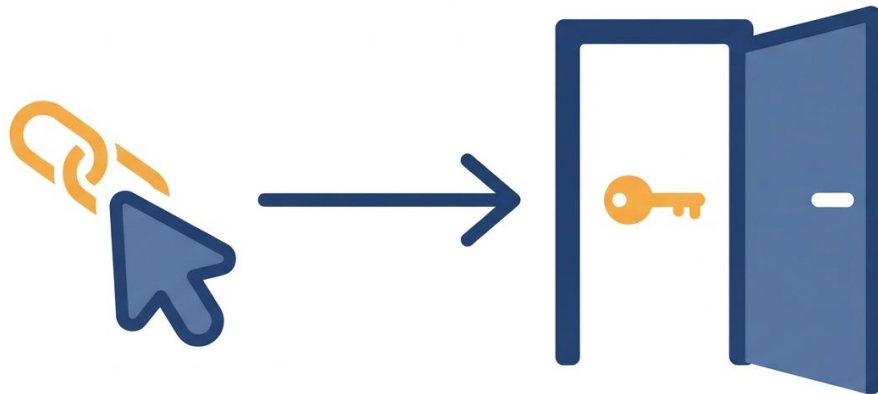


CASE

TikTok, 2022 - a one-click account takeover

Microsoft's security researchers found that a specially crafted link could force TikTok's Android app to load an attacker's web page inside its own privileged in-app browser - a browser that was connected to sensitive account-management features. A single click on that link could silently take over the victim's account. Microsoft disclosed it responsibly in February 2022, and TikTok shipped a fix within weeks.

[Microsoft Security Blog: TikTok Android one-click account takeover](#)





8 / 8

Putting It All on One Checklist

We've covered seven different kinds of problems. How do you make sure a brand-new app hasn't missed something? Security teams use a simple, memorable checklist called STRIDE.

● **S** - Spoofing: can someone pretend to be a real user or device?

● **T** - Tampering: can the app or its data be secretly changed?

● **R-I-D-E**

Repudiation: can a user deny that they performed an action because there is no reliable record?

Information Disclosure: can sensitive information be exposed to unauthorized people?

Denial of Service: can someone make the app or service unavailable?

Elevation of Privilege: can someone gain higher permissions than they are supposed to have?

S		☐
T		☐
R		☐
I		☐
D		☐
E		☐



From Theory to Practice

One axiom ties all eight ideas together: the client runs in an environment you don't control - so verify on the server, not the phone.

● Attack chains & tampering (1, 2)



● Threat modelling up front (8)

● Runtime manipulation (3)



● Server-side re-checking, not client "yes/no" flags

● Reverse engineering (4)



● Raise attacker cost - not a silver bullet

● Root/jailbreak evasion (5)



● Hardware-backed attestation (6)

● Unverified return links (7)



● Domain-verified App Links / Universal Links



Next: We Do This Ourselves

Session 2 (11:30) takes these same eight ideas hands-on - on real, safe-to-break practice apps, finding and fixing issues ourselves.

30-minute break

Session 1 ends on time at 11:00. Session 2 - “Advanced Security Analysis” - begins at 11:30 and runs the same eight ideas as a live, hands-on walkthrough.

That's a wrap